University Management System

Design, implement and analyse a small but fully working Python application.

The scenario

A fictional university needs a simple system for keeping student and module records, managing enrolments, recording attendance and marks, and producing useful academic reports. You are the developer. Your application should work without a graphical interface or database.

What your system must do

01	**Student and module records** Register, display, search and update students (ID, name, year, department) and modules (code, title, tutor). Reject duplicate IDs.
02	**Enrolment management** Enrol students on modules, show enrolment lists, and handle unknown student/module IDs and repeated enrolments sensibly.
03	**Attendance and assessment** Record weekly attendance and a final mark (0–100). Calculate attendance percentages, course averages and student summaries. Keep a missing mark distinct from a mark of zero.
04	**Rules and alerts** Assign example pass/fail labels and flag low attendance using clear, configurable rules (e.g. pass ≥ 40, attendance alert < 75%). These are fictional thresholds, not official university policies.
05	**Management dashboard** Show counts, mean/median marks, missing/duplicate-record checks and comparisons by module or year. Present a bar chart, scatter plot and time-based line chart.
06	**Research and evaluation** For one chosen module, use one observation per student to investigate attendance versus marks with Pearson correlation, and attendance category versus pass/fail with a chi-square test. Explain the findings and limitations.

Data, deliverables and boundaries

Use invented data only. Create at least 80 fictional students and three modules, including at least 50 distinct students in the module selected for statistical analysis. A fixed random seed is recommended. Include realistic variation and some missing/duplicate records for testing, but retain an unchanged raw dataset and a separate cleaned copy.

Submit one runnable Jupyter notebook (.ipynb) and the small .py files required for your own module/package. Do not add a GUI, database or machine learning model. Use the checklist on page 2 to verify coverage.

Please also provide a brief report summarizing your understanding (one to two pages), along with a 5-10 slide presentation. During the first session, you will begin working on your project and have the opportunity to ask any questions.

In the following week's session, you will be asked questions about your project to assess your understanding of the implementation. These questions will focus on how your code works, why you chose specific approaches or functions, and whether alternative solutions could have been used.

Your mark will be based on the quality of your answers, your understanding of the code, and your ability to explain and justify your implementation decisions.

Alongside this file, you will find Python 1 to 4.pdf and Correlation-2026.pdf, which are also useful for a quick review.

Required Python features

Tick each item only after it appears at least once in working code or a supported analysis. Prefer meaningful uses within your system.

SESSION 1 / PYTHON FOUNDATIONS

- □ Variables using str, int, float and bool; conversions and f-strings.
- □ Multiple assignment, arithmetic, operator precedence and updates (+=).
- □ Comparisons, and/or/not, if, elif and else decisions.
- □ A list with indexing, slicing, editing and append().
- □ A tuple and tuple unpacking (e.g. a timetable slot).
- □ A dictionary with lookup, insertion and updating.
- □ A set to remove duplicate items or collect unique values.
- □ A for loop using range() and enumerate().
- □ A while loop, with a meaningful break and continue.
- □ A reusable function with parameters and return.
- □ A default argument and a function returning multiple values.
- □ Type hints, docstrings, comments and clear PEP 8 names.

SESSION 2 / OBJECT-ORIENTED DESIGN

- □ A class with __init__, self, instance attributes and an instance method.
- □ A class variable and a @classmethod.
- □ A @staticmethod used for a calculation or validation.
- □ zip() to process paired data (e.g. IDs and names).
- □ A list comprehension with filtering and an if/else expression.
- □ A custom .py module imported from your own package.
- □ Randomly generated test data with a fixed seed.

WORKING DEMONSTRATION AND SUBMISSION

- □ At least 80 fictional students, 3 modules and 50 unique students in the analysis module.
- □ One notebook plus required package files, runnable from a fresh start.

SESSION 2 / NUMPY AND PANDAS

- □ NumPy 1D array with indexing and slicing.
- □ NumPy 2D array, reshape() and an independent copy.
- □ A NumPy Boolean mask for filtering.
- □ A pandas Series using meaningful index labels.
- □ A DataFrame with at least one calculated column.
- □ Selection using both .loc and .iloc.

SESSION 3 / EXPLORATION AND VISUALISATION

- □ Inspect head/tail, shape and data types.
- □ Count and handle missing values without treating unknown marks as zero.
- □ Detect duplicates; clean a copy while keeping original data.
- □ Compute counts, mean, median and a filtered summary.
- □ Use groupby() with an aggregation and compare groups.
- □ Produce labelled bar, scatter and time-series line plots.
- □ Use Matplotlib; add a Seaborn visual as well.

SESSION 3 / STATISTICAL REASONING

- □ State H_0, H_1, a chosen α and the p-value decision rule.
- □ Calculate sample covariance manually and verify it in pandas/NumPy.
- □ Report Pearson r, p-value and complete-pair sample size.
- □ Calculate and inspect a numerical correlation matrix.
- □ Check one potential confounder by comparing subgroups.
- □ Build a categorical crosstab and calculate expected counts.
- □ Calculate chi-square manually and check it with SciPy.
- □ Report chi-square, degrees of freedom, p-value and test limitations.
- □ Discuss strength, significance, assumptions and why association is not causation.